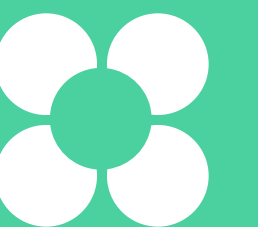


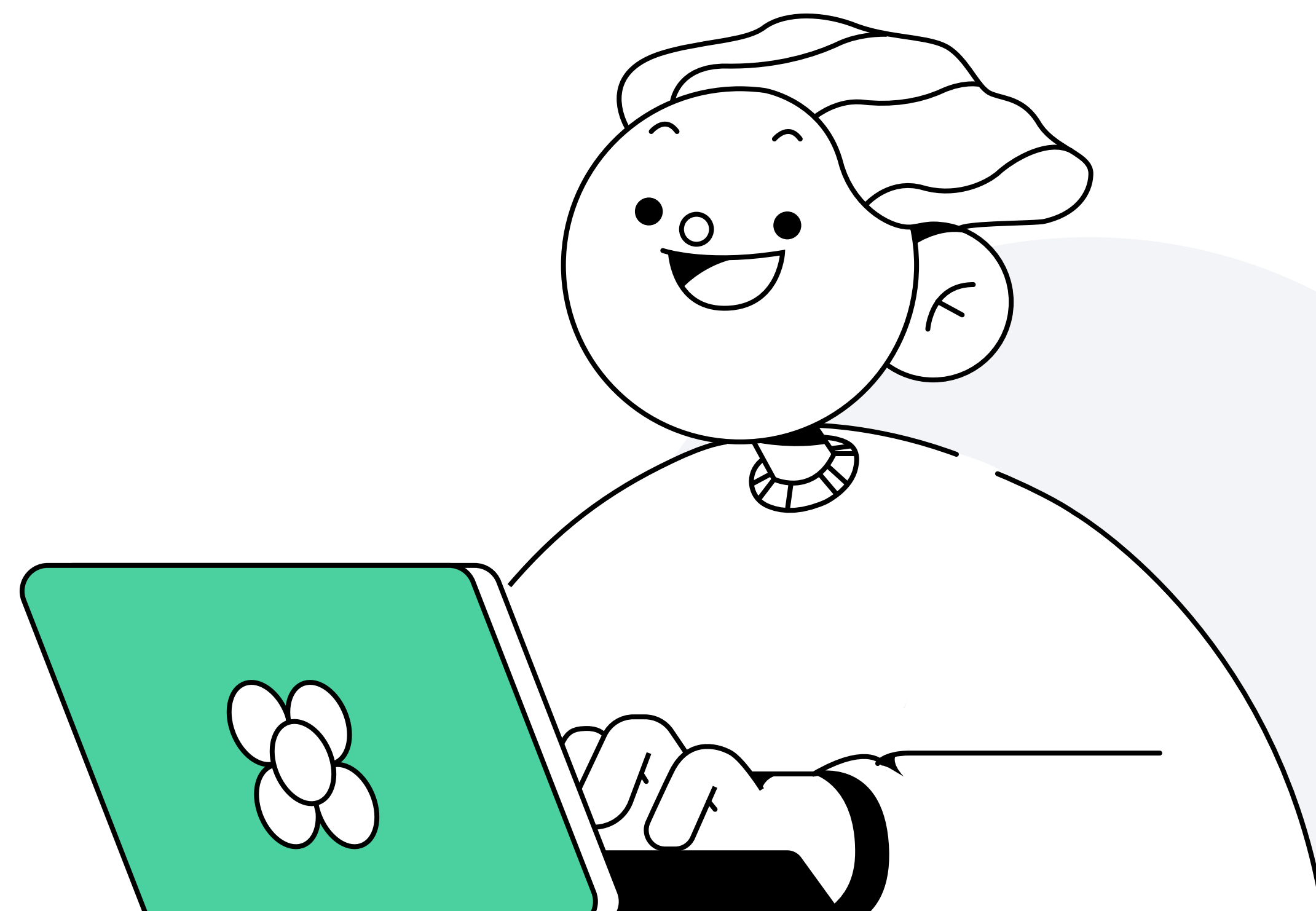
Тестирование roles

Алексей Метляков
Tech Lead DevOps engineer в OpenWay



План занятия

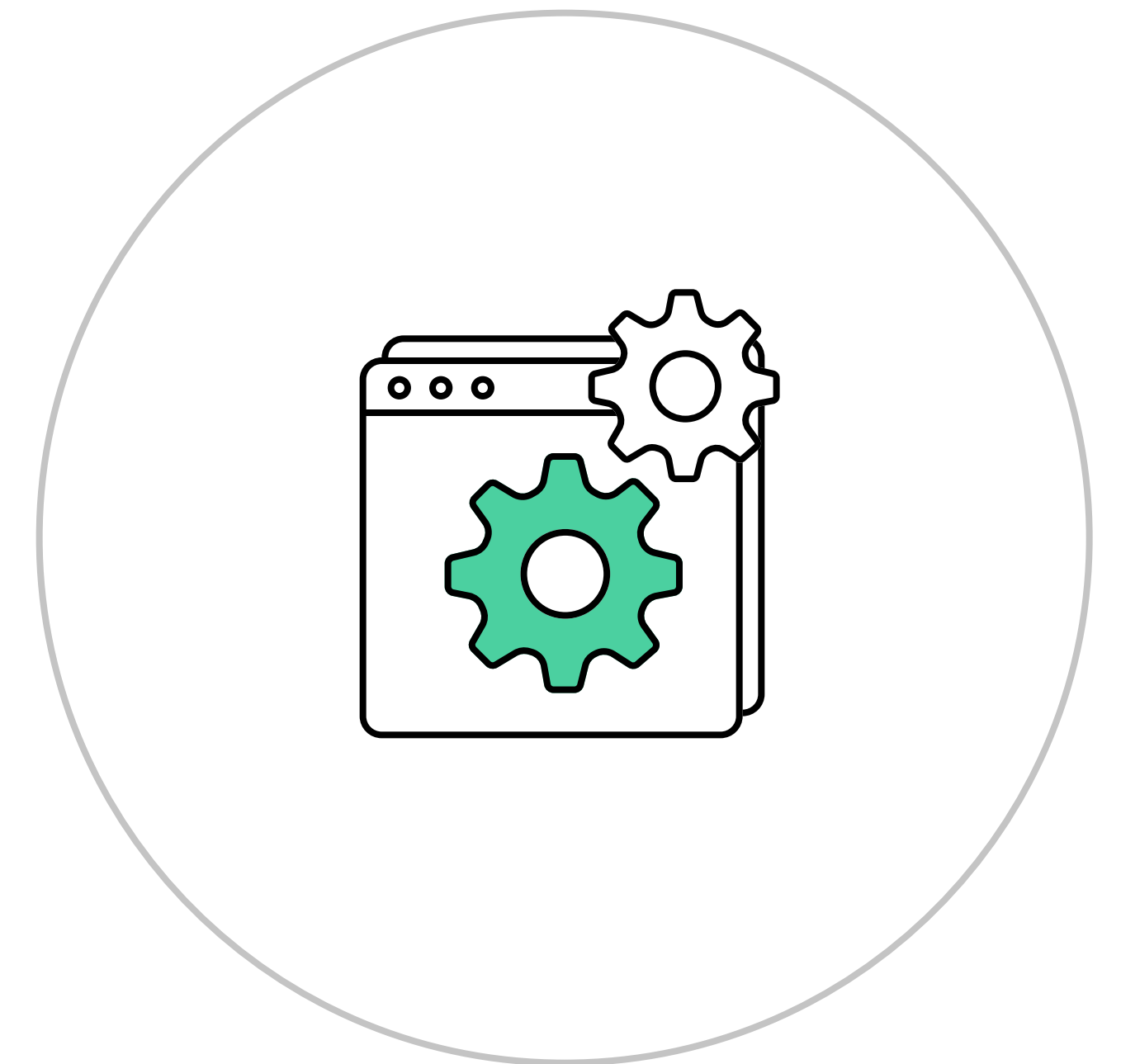
- 1 Molecule
- 2 Сценарии тестирования
- 3 Tox



Как тестировать role

Процедура тестирования

- Провести проверку синтаксиса
- Подготовить тестовое окружение
- Провести проверку на работоспособность
- Провести проверку на идемпотентность
- Исправить ошибки на каждом из этих этапов и повторять весь сценарий, пока не будет получен положительный результат



Molecule

Алексей Метляков

Tech Lead DevOps engineer в OpenWay

Molecule

Этот фреймворк позволяет избавиться нас от рутины и заниматься только созданием и исправлением ошибок. Он умеет:

- создавать новые `roles`
- создавать `scenarios` тестирования
- тестировать `roles` против разного окружения
- поддерживает `docker`, `podman`, `delegated` как драйверы подключений

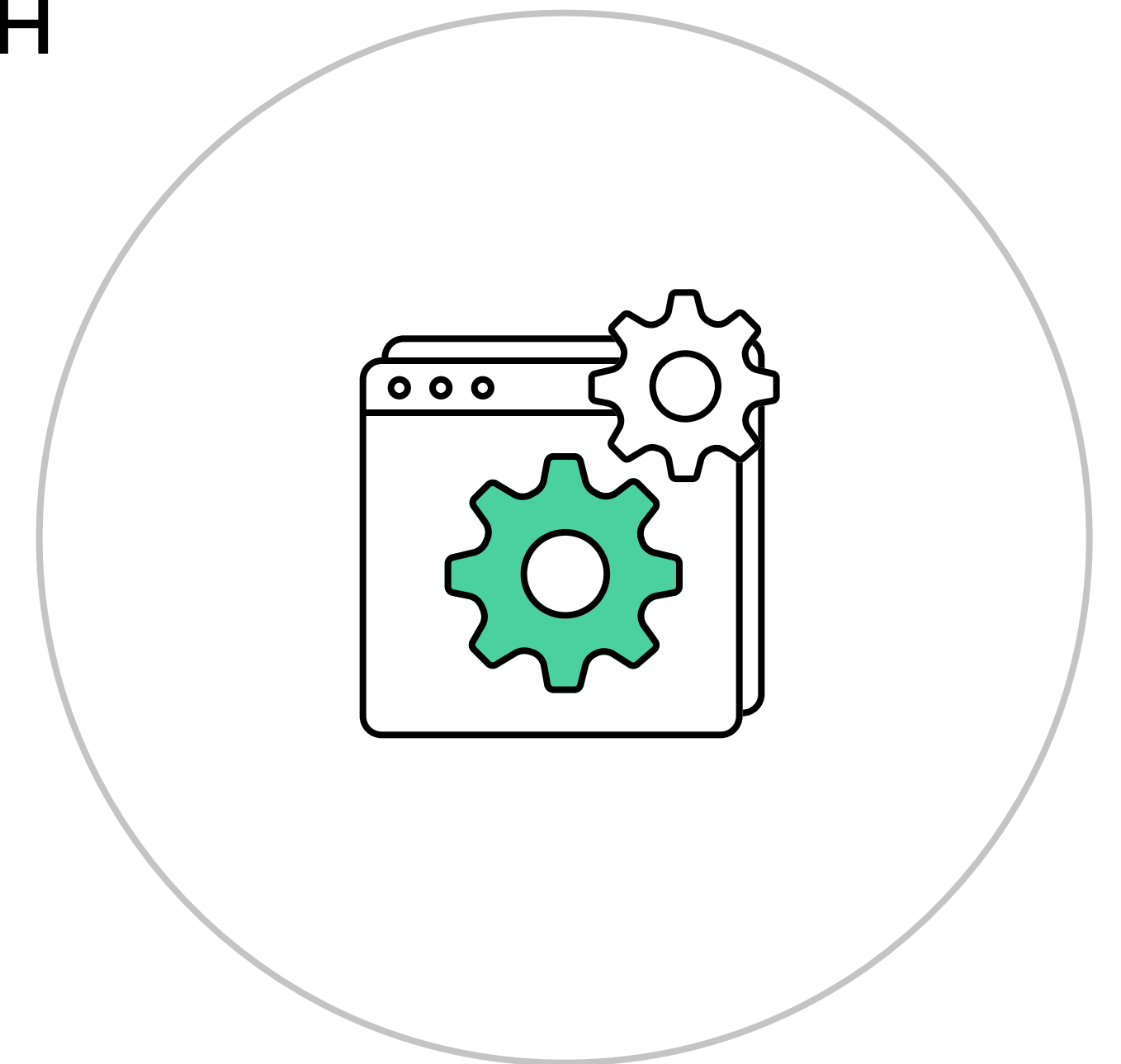
[Официальная документация](#)



Как правильно тестировать role

В этом виде тестирование role упрощается, так как нужно:

- подготовить тестовое окружение
- запустить сценарий проверки через molecule
- исправить ошибки на каждом из этих этапов и повторять весь сценарий, пока не будет получен положительный результат



Как создать role

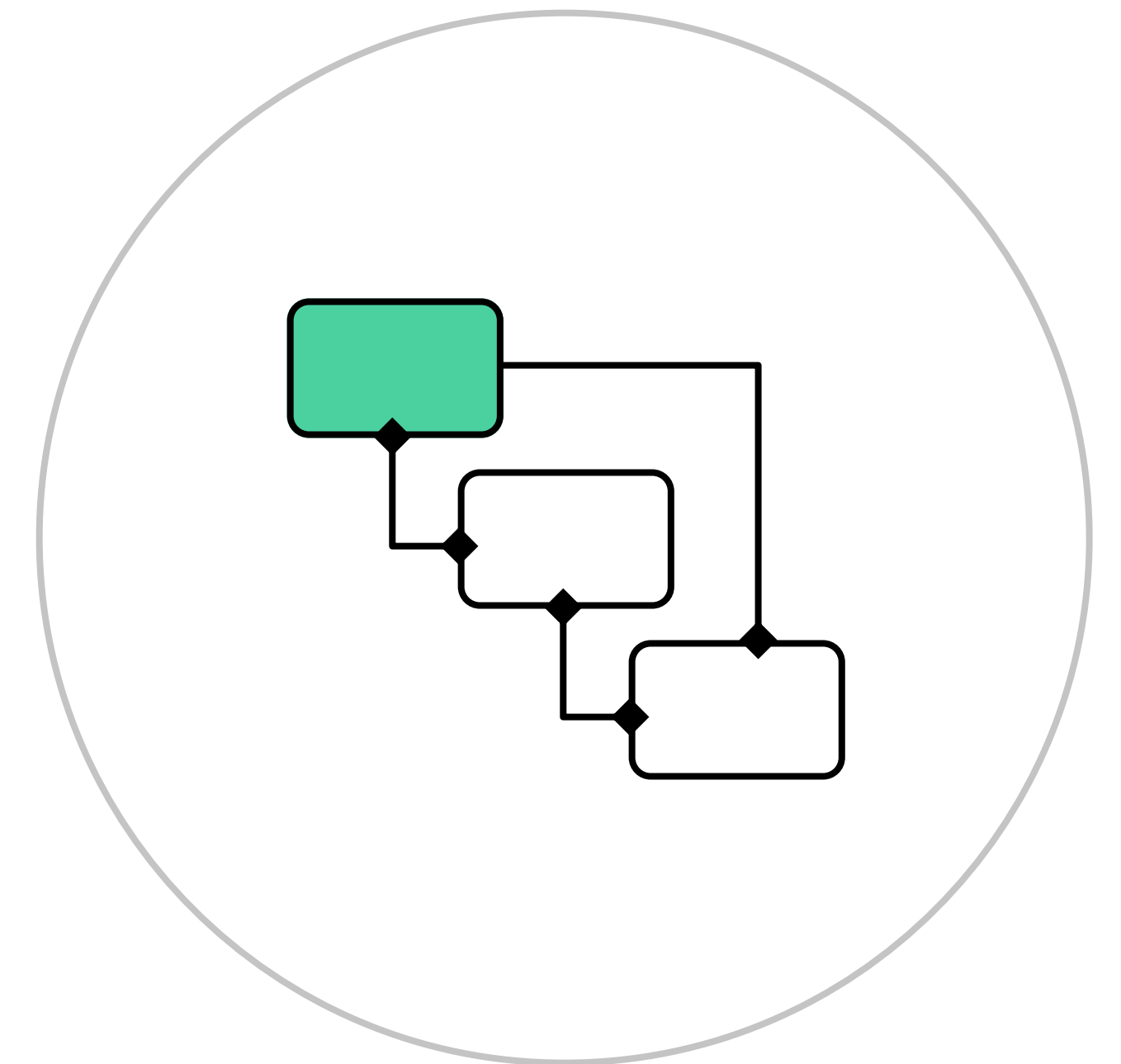
- Создать стандартную структуру директорий и файлов при помощи **molecule**
- Создать необходимые **tasks, handlers**
- Определить все необходимые переменные в **defaults** и **vars**
- Создать готовый тестовый **playbook** в **tests**, заполнить файлы **molecule** для проведения тестирования
- Заполнить в **meta** всю информацию о роли, наиболее полно описать её в **README.md**



Структура директорий

После инициализации новой роли мы получаем следующие директории и файлы:

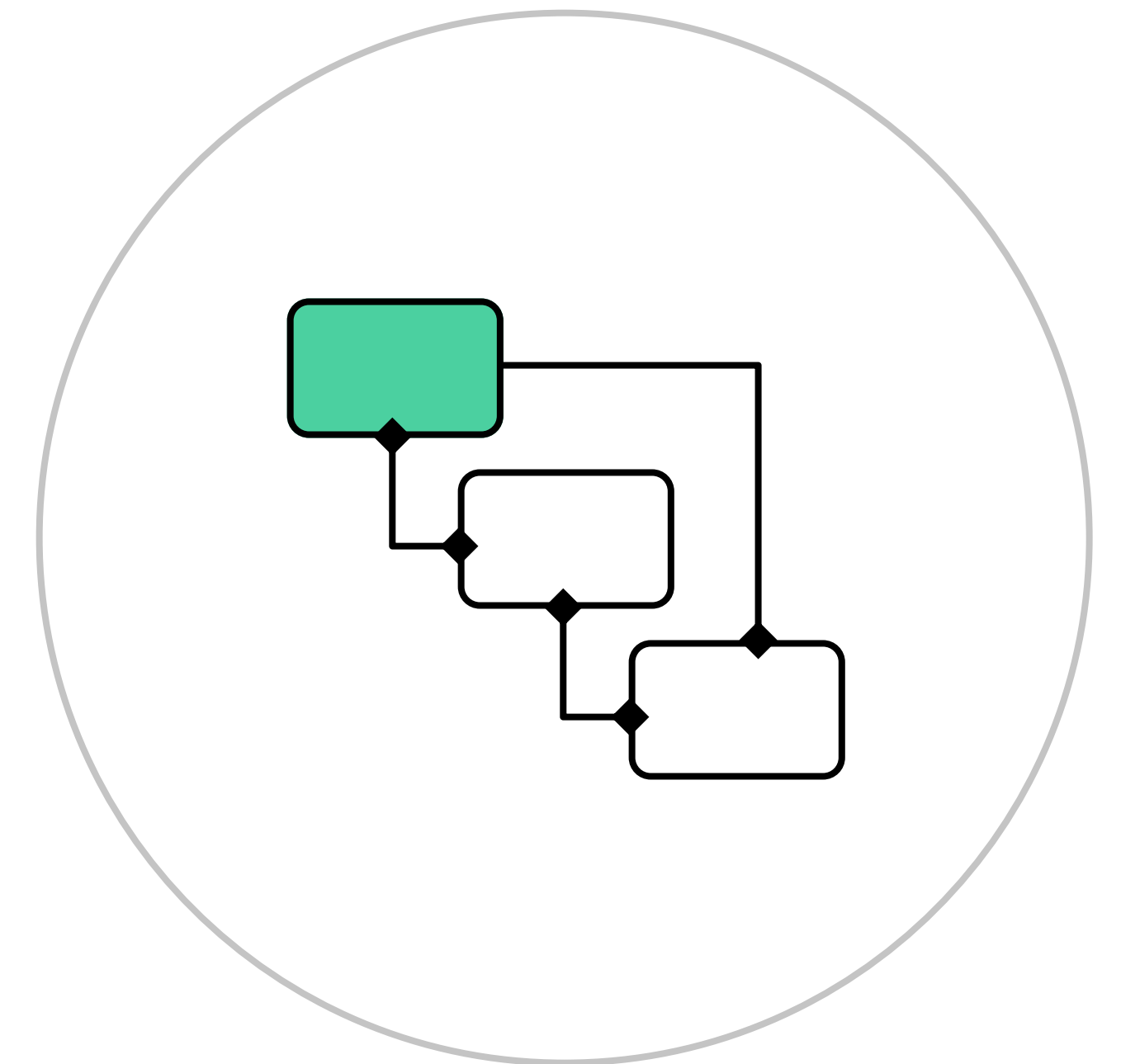
- стандартный набор директорий и файлов для `role`
- `molecule` – набор `scenarios` для тестирования



Структура директорий

Внутри любого scenario находятся следующие файлы:

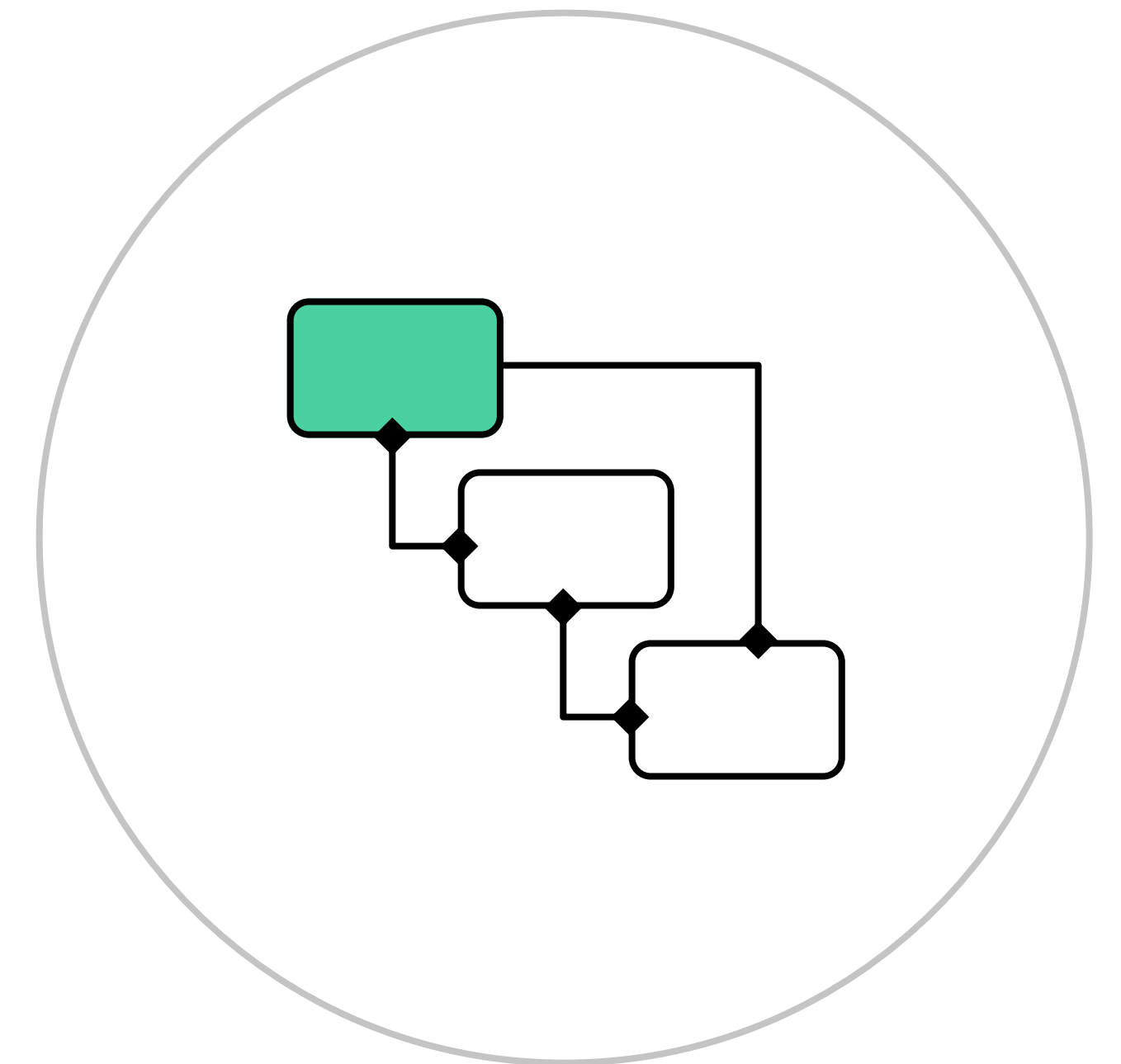
- `molecule.yml` – основной файл для molecule



Структура директорий

Внутри любого scenario находятся следующие файлы:

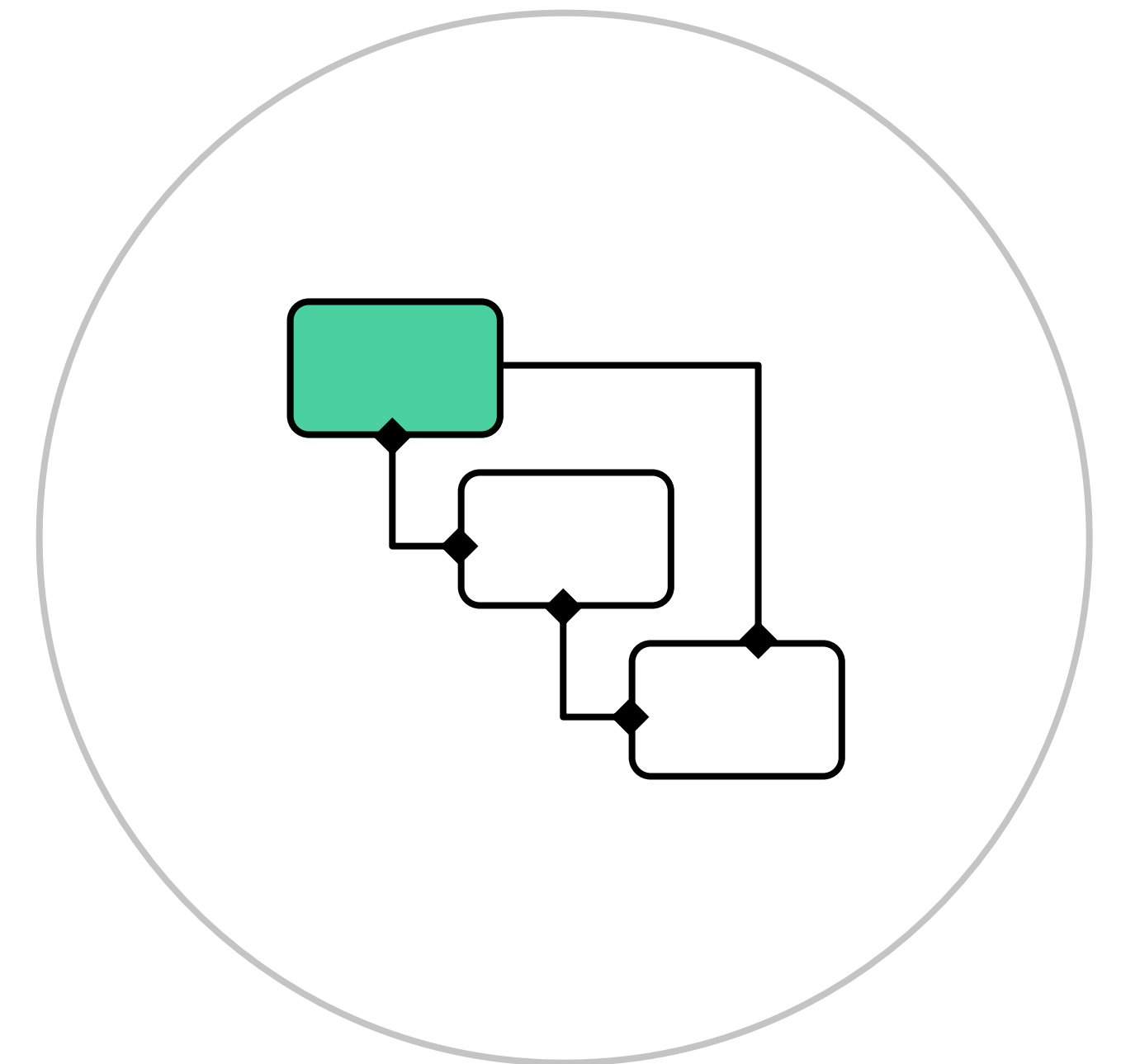
- `molecule.yml`
- `converge.yml` — playbook для запуска тестов



Структура директорий

Внутри любого scenario находятся следующие файлы:

- molecule.yml
- converge.yml
- verify.yml — дополнительные тесты после исполнения role



Структура molecule.yml

Внутри файла находятся следующие директивы:

- **dependency** — перечисление зависимостей роли
- **driver** — указание параметров выбранного driver
- **platform** — перечисление хостов для выбранного driver
- **provisioner** — указание поставщика для molecule
- **verifier** — выбор framework для проведения проверок

Туда же можно добавить:

- **lint** — конфигурирование linter для тестирования
- **scenario** — перечисление сценариев тестирования



Сценарии тестирования

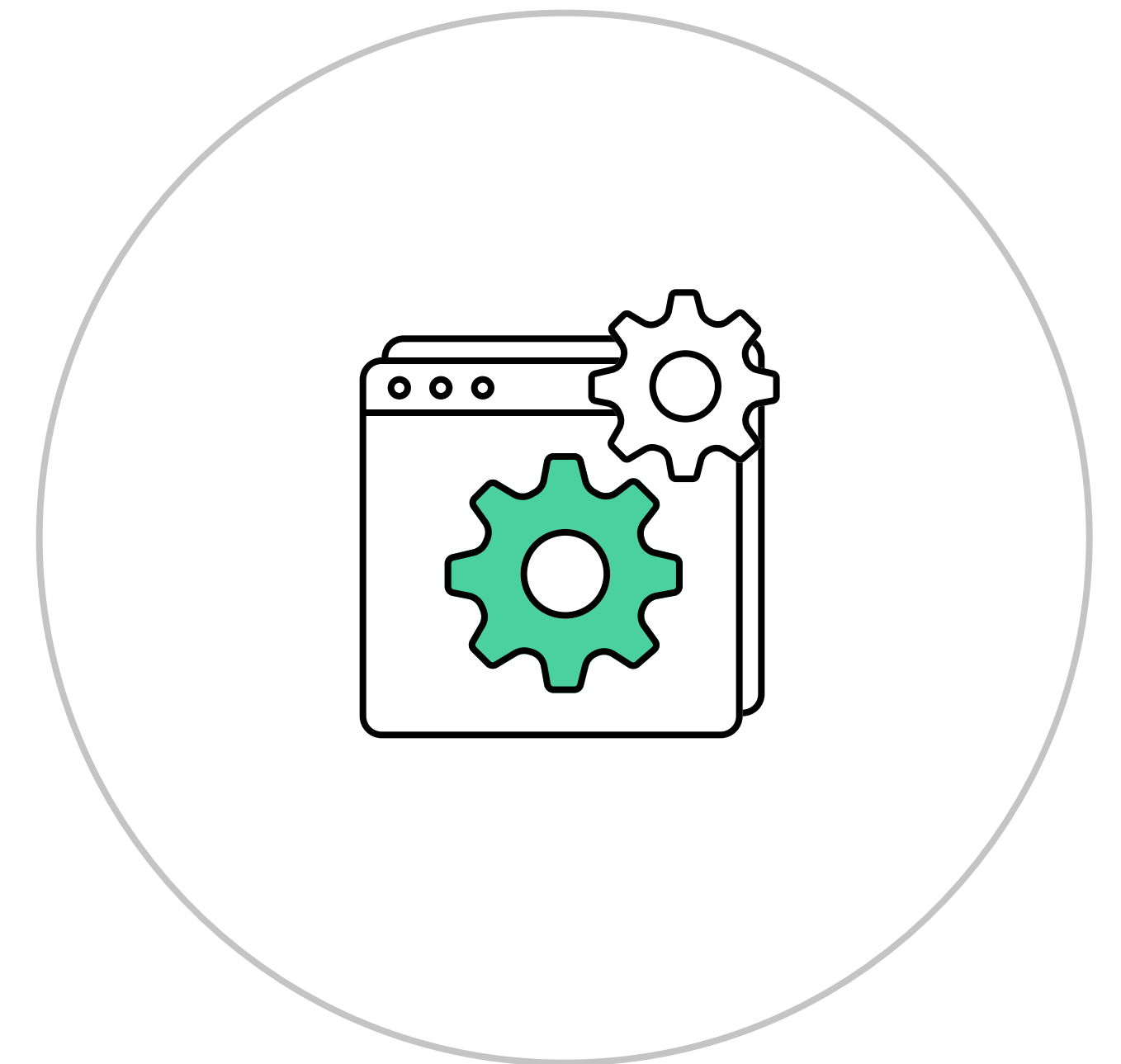
Алексей Метляков

Tech Lead DevOps engineer в OpenWay

Как правильно тестировать role

Тестирование условно можно разделить на три вида:

- 1) Использование полного сценария тестирования
- 2) Использование собственных сценариев тестирования
- 3) Использование отдельных частей сценария самостоятельно

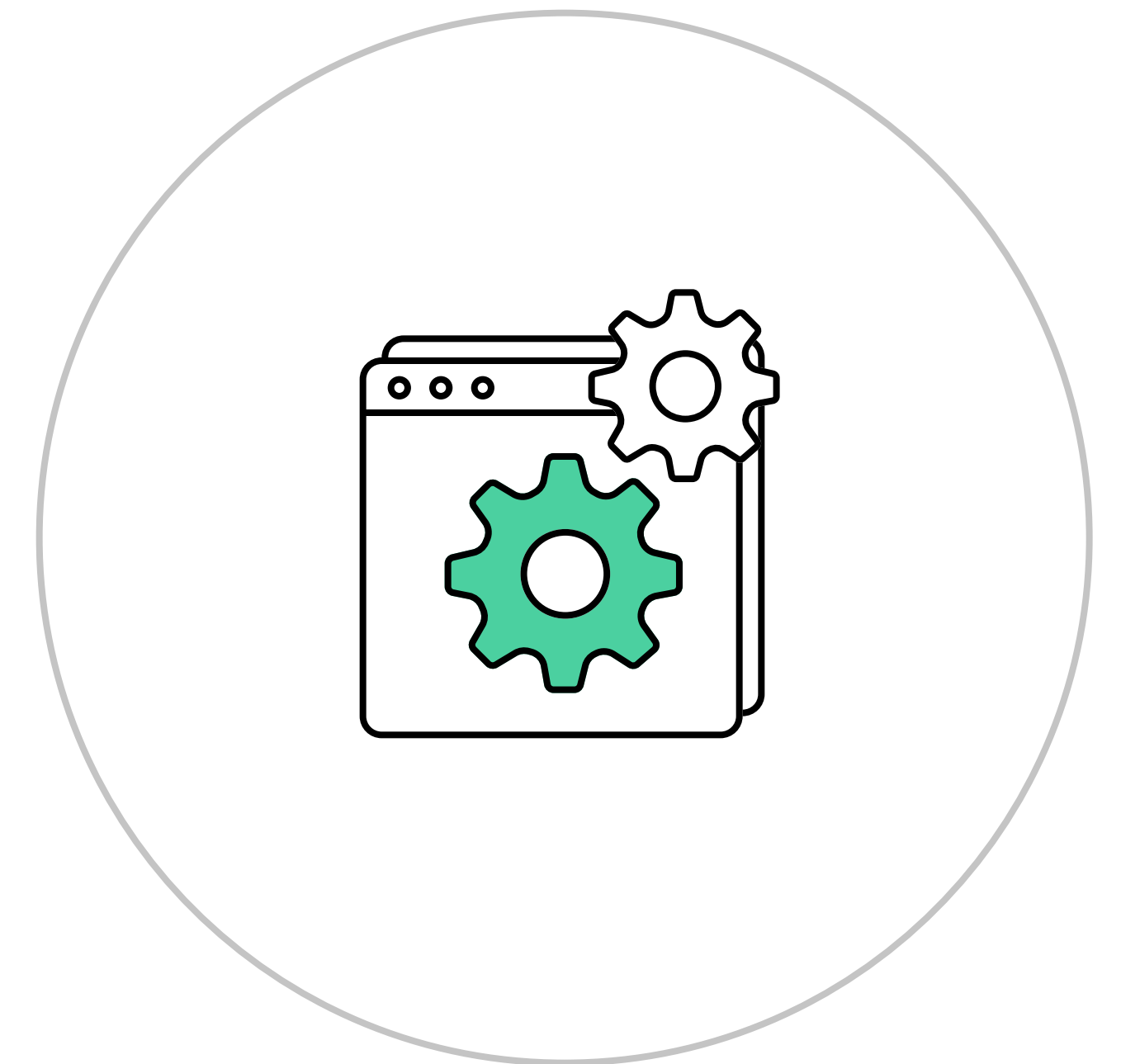


Как правильно тестировать role

Molecule test запускает полный сценарий тестирования role.

Полный сценарий включает в себя:

- `lint` — прогон линтеров
- `destroy` — удаление старых инстансов с прошлого запуска
- `dependency` — производит установку Ansible-зависимостей
- `syntax` — проверка синтаксиса с помощью `ansible-playbook --syntax-check`
- `create` — создание инстансов для тестирования
- `prepare` — подготовка инстансов, если это необходимо

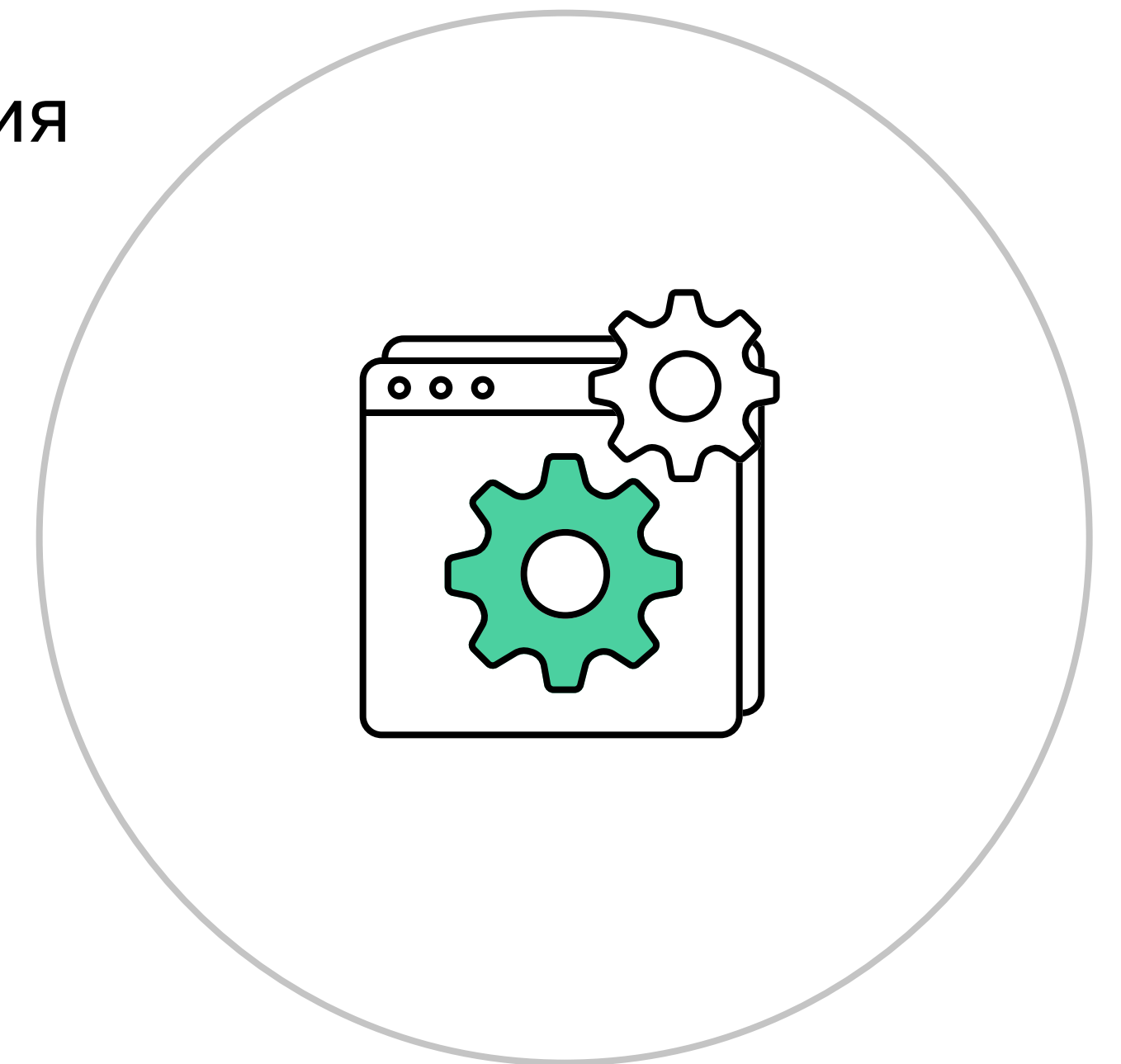


Как правильно тестировать role

Molecule test запускает полный сценарий тестирования role.

Полный сценарий включает в себя:

- **converge** — запуск тестируемого плейбука
- **idempotence** — проверка на идемпотентность при помощи повторного запуска
- **side_effects** — действия, которые не относятся к role, но необходимы для тестирования
- **verify** — запуск тестов с помощью указанного фреймворка тестирования
- **cleanup** — очистка внешней инфраструктуры от результатов тестирования
- **destroy** — уничтожение инстансов для тестирования



Как правильно тестировать role

- Каждую из указанных частей сценария можно вызвать отдельно, но нужно держать в уме, что у каждой из них могут быть зависимости
- Чтобы понимать, что каждая из tasks будет запускаться в случае отдельного вызова, необходимо пользоваться конструкцией `molecule matrix <task_name>`
- Перед тем, как уничтожать инстансы, к ним можно подключиться и в ручном режиме проверить все изменения в системе
- Оставить возможность подключения можно и с полным сценарием тестирования, воспользовавшись параметром `--destroy=never`



Как правильно тестировать role

Очерёдность сценариев можно переопределить через директиву `scenario`. Формат записи в `molecule.yml` будет выглядеть так:

```
---
scenario:
  <task>_sequence:
    - list
    - of
    - tasks
...
---

#Example of redefined of test scenario
scenario:
  test_sequence:
    - create
    - converge
    - idempotence
    - destroy
...
```

Tox

Алексей Метляков

Tech Lead DevOps engineer в OpenWay

Tox

Этот фреймворк позволяет проводить тестирование любого Python кода, грубо говоря, Tox — менеджер виртуальных окружений.

- Выбирать, какие версии Python использовать — версии должны быть установлены в системе
- Выбирать, какие дополнительные модули должны быть установлены
- Тестировать разные версии модулей друг против друга — создавать матрицы тестирования

[Официальная документация](#)



Установка Tox

Так как Tox – библиотека Python-кода (как и Ansible), то его установка происходит достаточно просто:

- `pip3 install tox`
- `pip install tox`



Настройка Tox

В нашем случае необходимы два файла для использования Tox:

- **tox.ini** — основной файл настройки, содержит в себе перечисление Python и возможных модулей плюс описание их перечисления друг против друга, а также указание команды, которую необходимо выполнить для проведения тестирования
- **test-requirements.txt** — перечисление дополнительных модулей, которые не должны иметь итерирования против разных версий Python



Запуск Tox

- `tox` — запуск на всех возможных окружениях
- `tox -l` — показ всех возможных окружений
- `tox -r` — принудительное пересоздание окружений
- `tox -e <env_names>` — запуск на указанных окружениях

